

TEST PLAN

Multi-Modal Agentic AI System for Medical Diagnosis and Triage

Author: Karan | Role: Test Plan Manager

Team GitHub Repo: github.com/abhishek7112000/Week-2-Team-9

1. Purpose and Scope of Testing

This Test Plan defines how the team verifies and validates the system against Pipeline Workflow v2.0: each pipeline stage is checked in isolation and end-to-end, every KPI is evidenced, and clinical safety is protected by minimising missed serious conditions.

The pipeline is: data pre-processing → model training → evaluation → ONNX export → agent deployment.

Stage	What is tested	Test type
Data pre-processing & feature engineering	Loaders, splits, image/tabular/text transforms (3 datasets).	Data validation, unit
Model training & tuning (M1–M3)	Training loops, hyperparameter tuning, reproducibility.	Unit, integration
ONNX export & quantisation	PyTorch↔ONNX output parity; INT8 size within Vercel limit.	Unit, integration
Model evaluation	Metrics vs KPIs for all three models.	Model performance
Agent (LangGraph.js) + MedGemma + RAG	Node routing, tool-selection, JSON validity, triage decision.	Unit, integration, safety
Next.js app + Calendar/Gmail	API routes, end-to-end inference, booking, usability.	Integration, UAT

Out of scope: clinical certification, real patient data, internals of third-party weights (MedGemma, OpenMed base), and production load/penetration testing. MedGemma-27B-IT is the agent's reasoning layer, not one of the three assessed ML models, so it is tested for output validity and safe behaviour, not trained/evaluated as a model.

2. Coding Standard

One enforced standard across both languages; all checks must pass in CI (pre-commit + GitHub Actions) before any merge.

Area	Python (model training)	TypeScript (app & agent)
Format & style	PEP 8, Black, isort.	Prettier, ESLint.
Lint & types	Flake8 / Pylint, mypy type hints.	TypeScript strict mode, ESLint.
Naming	snake_case funcs/vars, PascalCase classes.	camelCase funcs/vars, PascalCase components/types.
Docs	Google-style docstrings.	JSDoc/TSDoc; root + module READMEs.
Errors & logging	Specific exceptions; logging module; no secrets/PII logged.	Typed errors; fail-safe to ONNX outputs; no secrets logged.
Version control	Protected main; feature branches; reviewed PRs; imperative commit messages (both languages).	

3. Data Quality Testing / Validation

- **NIH ChestX-ray14 (M1):** files open and are non-corrupt; resized to 224×224, greyscale→3-channel, ImageNet-normalised; labels within the 14 known classes; official patient-level split gives zero leakage; class balance logged.

- **Synthea EHR (M2)**: schema and dtypes valid; filtered to chest/cardiac ICD-10; vitals/labs within plausible ranges; missing-value strategy with is_imputed flags; duplicates removed.
- **Synthea NER set (M3)**: valid BIO tagging from ICD-10/SNOMED distant supervision; patient-level 80/10/10 split.
- **Encyclopedia / RAG**: chunked (512-token, 64 overlap); embeddings non-empty; known-condition look-ups return expected chunks.
- **Cross-cutting**: fixed seeds for reproducible splits; stratification preserves class distribution; no patient appears in more than one split.

4. Unit and Integration Testing

Functions are tested in isolation and run automatically in CI; target $\geq 80\%$ line coverage on /src. One integration test runs the full agent path (intake $\rightarrow$ ner $\rightarrow$ route $\rightarrow$ xray/clinical $\rightarrow$ synthesise $\rightarrow$ rag $\rightarrow$ triage $\rightarrow$ end).

Component / function	Example test	Tool	Expected
preprocess_image()	Arbitrary-size image in.	pytest	224×224×3 output.
Metric calc (AUC / F1 / sequeval)	Known inputs.	pytest	Correct value (AUC=1.0 perfect).
ONNX parity check	PyTorch vs ONNX, 100 samples.	pytest + onnxruntime	M1 $ \Delta < 1e-4$; M2 ≤ 0.01 ; M3 F1 $\Delta < 0.5pp$.
assign_triage()	Severity=3 / SpO ₂ <90%.	jest	Returns EMERGENCY.
Agent nodes (route/synthesise)	Mocked model outputs.	jest	Correct conditional edge taken.
API routes /api/models/*	Sample payload.	jest	Valid JSON of expected shape.
End-to-end flow	Symptoms + X-ray $\rightarrow$ report.	Playwright	Complete DiagnosisReport, no error.

5. Model Performance Evaluation Against KPIs

All three models are evaluated on the same held-out test set. Tuning uses Optuna (M1, M2 fusion) and GridSearchCV (XGBoost) on the validation set only; pre/post-tuning gains and an ablation (XGBoost-only / text-only / hybrid) are recorded. Each KPI passes only when its target is met.

Model / aspect	Metric & target (KPI)	Test method
M1 ChestVision (custom NN)	Mean AUC-ROC ≥ 0.82 ; per-class AUC ≥ 0.75 ; F1 ≥ 0.60 (top 5).	Test-set evaluation.
M2 ClinicalFusion (hybrid)	Macro F1 ≥ 0.80 ; weighted F1 ≥ 0.85 ; ROC-AUC ≥ 0.90 /class; severity MAE < 0.4 .	Test-set evaluation.
M2 — clinical safety (primary)	False-low rate (critical $\rightarrow$ low) $< 2\%$.	Test set + audit log.
M3 OpenMed NER (pre-trained)	Entity F1 ≥ 0.82 (≥ 0.80 /type); DISEASE recall ≥ 0.85 ; transfer gain ≥ 3 F1 pts vs base.	sequeval span-level eval.
ONNX deployment (all)	Quantised size within Vercel 250MB limit; output parity within tolerance.	Size + parity checks.
Latency (all)	M1 $< 50ms/img$; M2 $< 20ms/record$; M3 $< 30ms/note$ (CPU).	onnxruntime CPU benchmark.

6. Triage Safety and User Acceptance Testing

Triage safety: the four levels (EMERGENCY / URGENT / ROUTINE / SELF_CARE) must route to the correct action; EMERGENCY triggers A&E guidance plus a GP alert with no booking; on uncertainty the agent fails safe by escalating and never downgrades a serious finding. UAT runs before the demo and is signed off by the Team Leader once all critical scenarios pass.

ID	Scenario	Acceptance criterion
UAT-01	Upload X-ray + enter symptoms.	Diagnosis, severity and triage returned within target time.
UAT-02	Cardiac-suggestive case.	Cardiology referral, correct urgency, appointment booked.
UAT-03	Emergency case.	A&E guidance, GP email, no booking.
UAT-04	Invalid file uploaded.	Clear, friendly error; app recovers.
UAT-05	First-time user, guidance only.	Completes a full diagnosis unaided.

7. Code Review and Sign-off Process

Workflow: feature branch → pull request → CI green (format, lint, types, unit tests, ONNX parity) → peer review against the checklist (meets spec; follows coding standard; tests added and passing; data-safe; ONNX parity validated) → merge. No code reaches main without an independent review and a recorded sign-off.

Sign-off gate	Criteria to pass	Signed off by
Automated CI	All checks green (format, lint, types, tests, ONNX parity).	Automated
Peer code review	Checklist satisfied; comments resolved.	Victor (Code Review Mgr)
Test sign-off	Data, unit and model tests pass; KPIs met or improved.	Karan (Test Plan Mgr)
Release / UAT	All critical UAT scenarios pass.	Austin (Team Leader)